Proyecto de Software

Identidad y acceso en aplicaciones web

OAuth 2.0 · OpenID Connect · JWT

Qué vamos a poder explicar

- Distinguir **identificación**, **autenticación** y **autorización**.
- Reconstruir el acceso delegado de **OAuth 2.0**.
- Explicar cómo **OpenID Connect** resuelve el inicio de sesión federado.
- Leer un **JWT** y decidir si corresponde aceptarlo.
- Ubicar los permisos que siguen siendo responsabilidad de nuestra aplicación.

Una API también necesita controlar el acceso

En REST preguntábamos:

> ¿Qué operación, datos y respuesta acuerdan cliente y servidor?

Hoy agregamos:

> ¿Quién puede invocar esa operación y con qué autoridad?

Una API bien diseñada no queda protegida por usar JSON o HTTP.

¿Quién puede aprobar una solicitud?

Una aplicación del hospital gestiona **solicitudes de comidas**.

- Una persona inicia sesión mediante un proveedor de identidad.
- La aplicación reconoce a esa persona.
- La aplicación decide si puede **ver, solicitar o autorizar**.

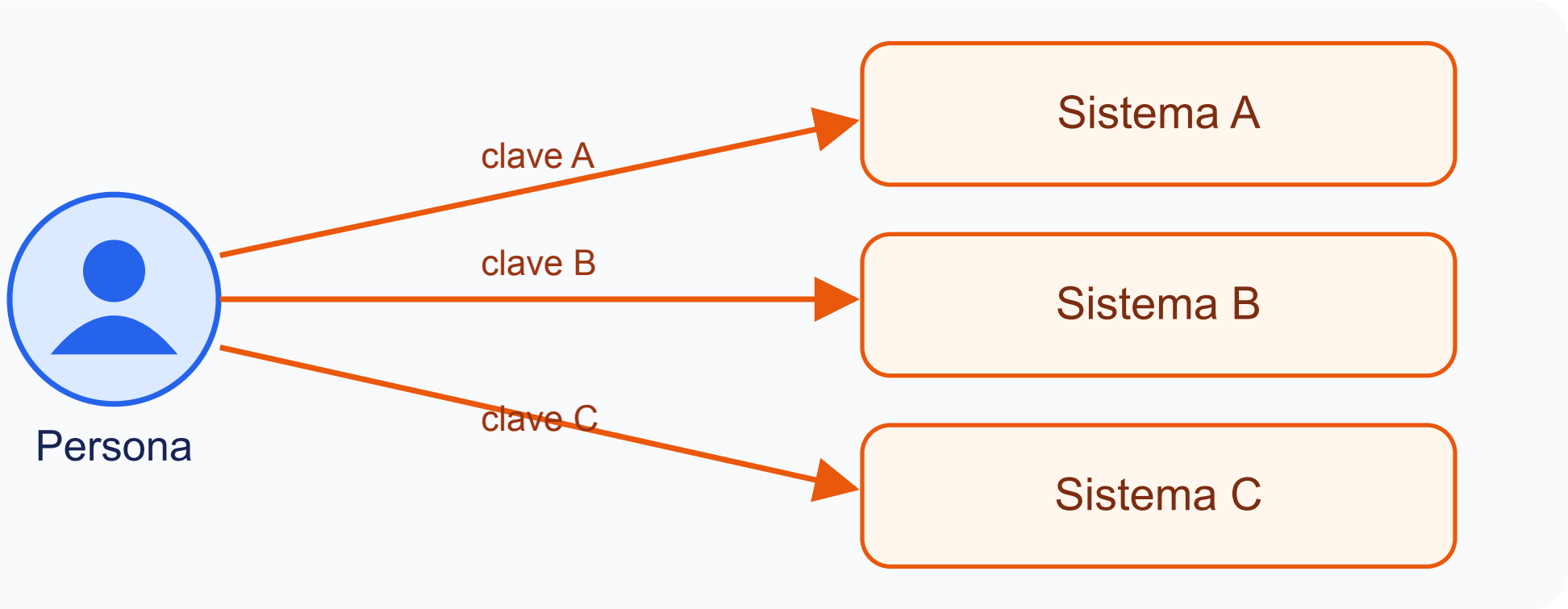
Ingresar al sistema no autoriza automáticamente a aprobar solicitudes.

Tres preguntas distintas



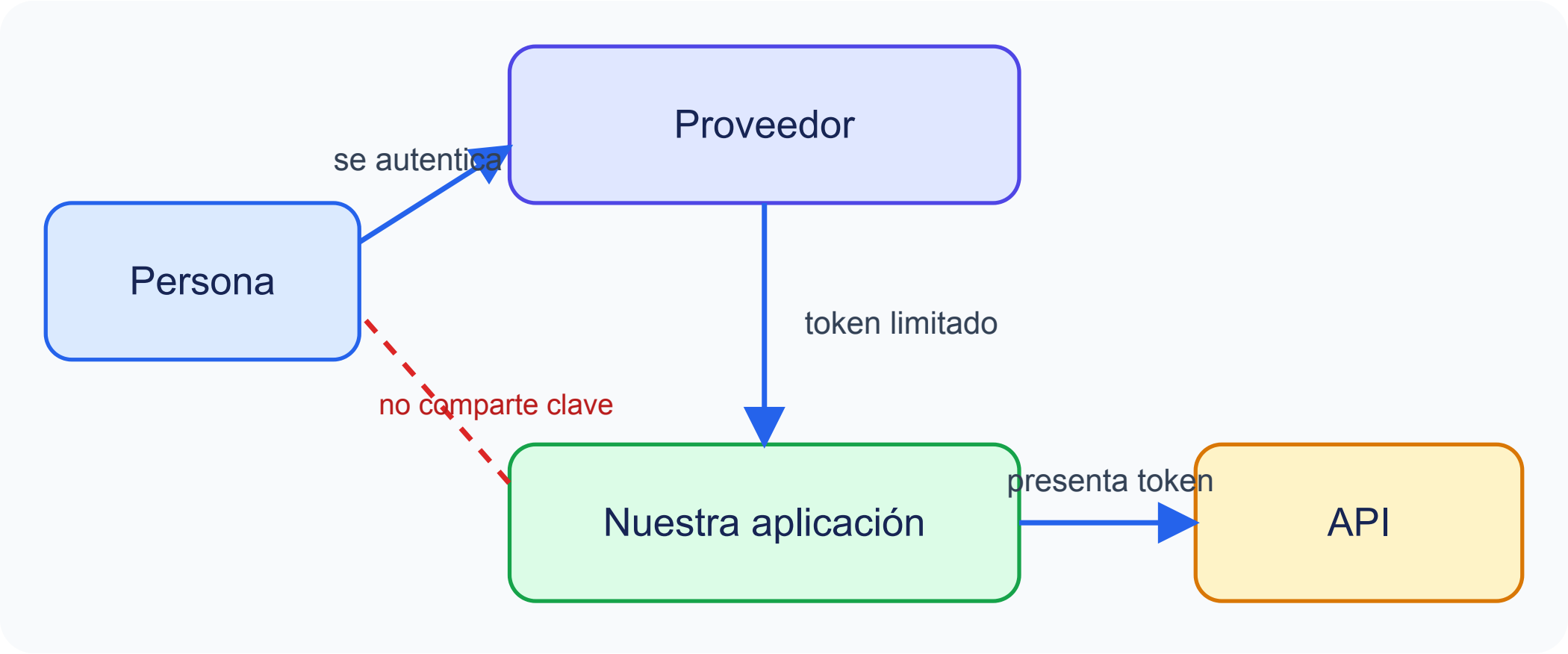
Estar autenticado no implica tener todos los permisos.

Antes: cada aplicación pide una contraseña



Más credenciales, recuperación y mantenimiento en cada sistema.

Delegar no es entregar la contraseña



La aplicación recibe una **capacidad acotada**, no la contraseña de la persona.

OAuth 2.0, en una oración

Marco para que un **cliente** obtenga acceso **limitado** a un recurso protegido, con intervención del dueño del recurso o una autorización previa.

OAuth trata de delegar acceso. No define, por sí solo, cómo iniciar sesión.

¿Quién participa en OAuth?

Rol	En el ejemplo
Resource owner	Persona que puede conceder acceso
Client	Nuestra app
Authorization server	Emite tokens tras autorizar
Resource server	API que protege los datos (en el ejemplo: Google Calendar)

El servidor de autorización y la API **pueden pertenecer al mismo servicio**.

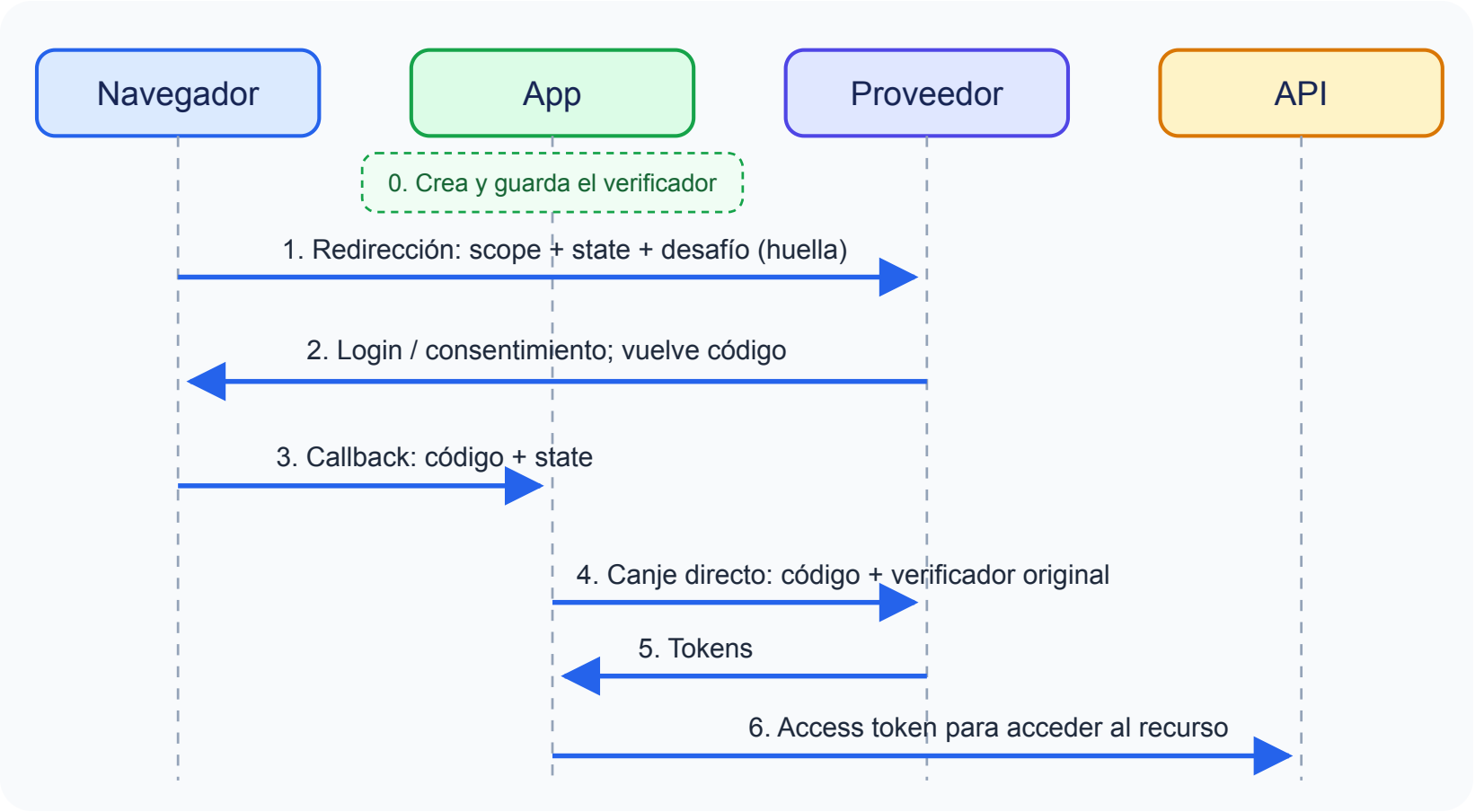
Los roles de OAuth no son permisos

Rol OAuth: `client`, `resource owner`, `authorization server`...

Rol de negocio: Jefe de Servicio, Alimentación, Comedor...

Un proveedor puede autenticar a la persona; **nuestra aplicación** sigue decidiendo si puede autorizar una solicitud.

Del navegador a la API



El código temporal no es un access token.

Paso 0: registrar el cliente

Antes del flujo, el proveedor conoce:

- `client_id`: identifica la aplicación; **no es un secreto**.
- `redirect_uri`: destino permitido para la respuesta.
- Métodos de autenticación del cliente, si corresponden.

La URI de retorno se configura de forma precisa: no se acepta cualquier destino.

En el ejemplo: `http://127.0.0.1:5004/auth/callback`

Paso 1: iniciar la autorización

```
GET /authorize?response_type=code
  &client_id=app-hospital
  &redirect_uri=https://app.example/callback
  &scope=openid%20email
  &state=aleatorio-por-intento
  &code_challenge=...
  &code_challenge_method=S256
```

Representación simplificada: el navegador sigue la redirección; la app no recoge la contraseña.

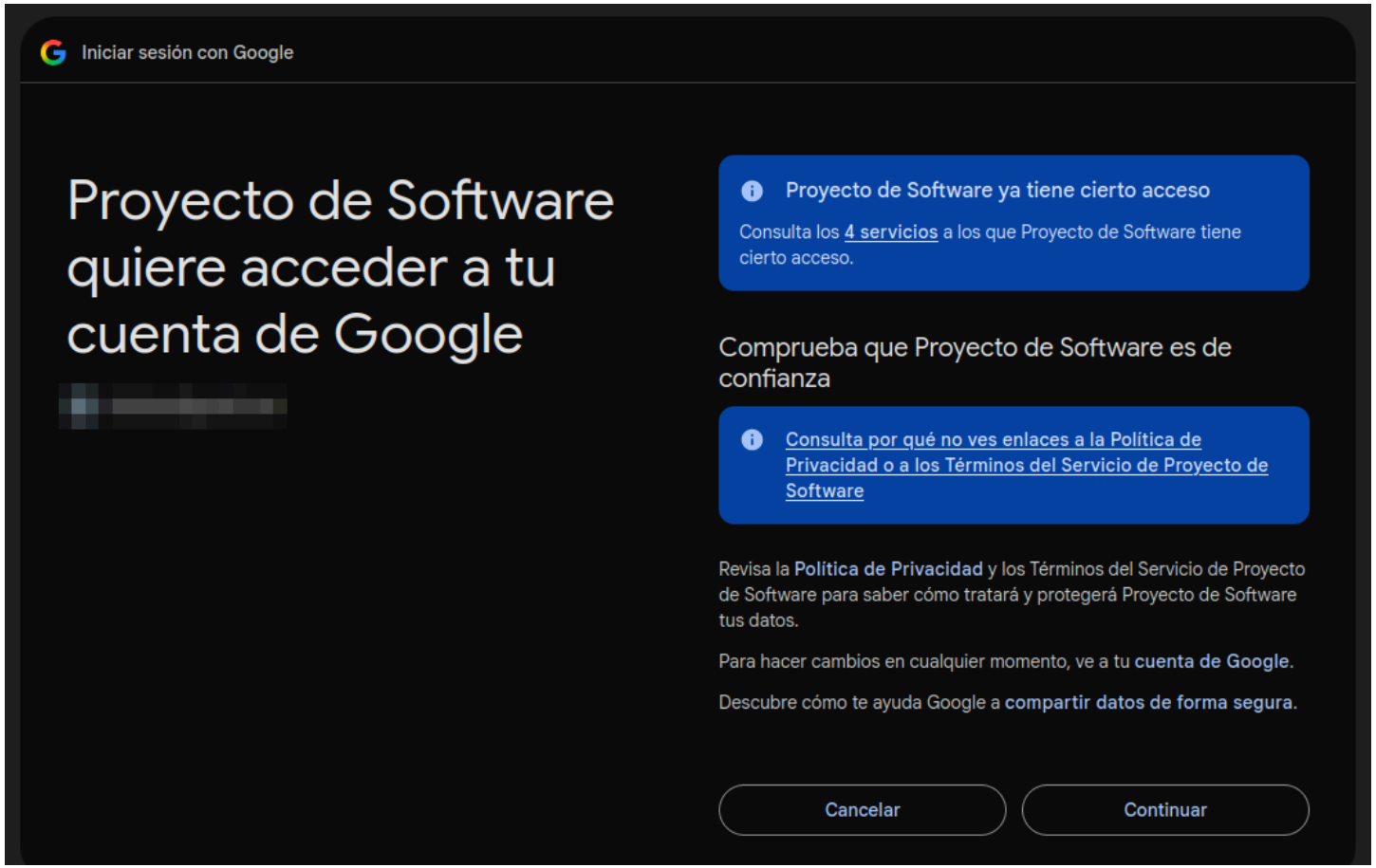
Scope: qué acceso se solicita

Un `scope` expresa un **alcance solicitado** al proveedor.

- Se piden sólo los alcances necesarios.
- Su significado concreto depende del proveedor y de la API.
- `openid` activa una solicitud **OIDC**.

Un scope externo no reemplaza los permisos internos del hospital.

Paso 2: ¿reconocen esta pantalla?



¿Quién pide acceso? ¿Quién decide? ¿Qué permiso no aparece?

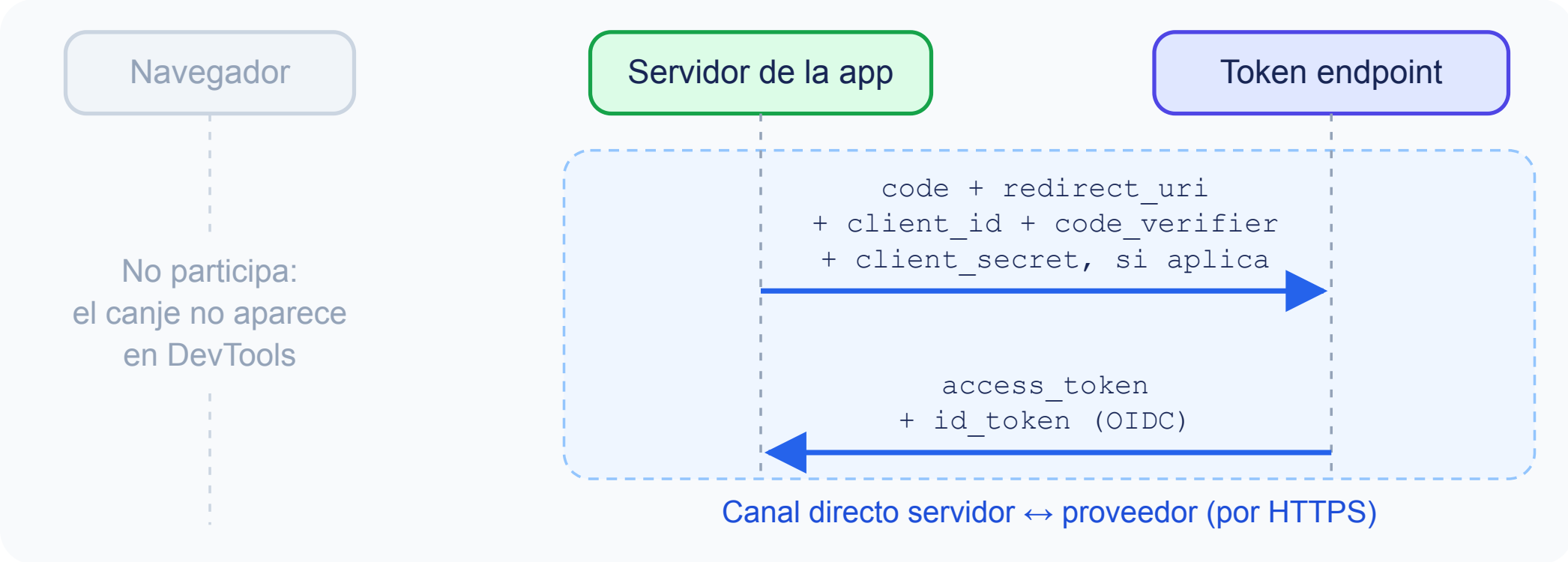
Paso 3: vuelve un código, no un token de API

```
GET /callback?code=CODIGO_TEMPORAL&state=aleatorio-por-intento
```

La aplicación comprueba que la respuesta corresponde al intento iniciado y luego intercambia el código con el proveedor.

El **authorization code** es breve y de un solo uso.

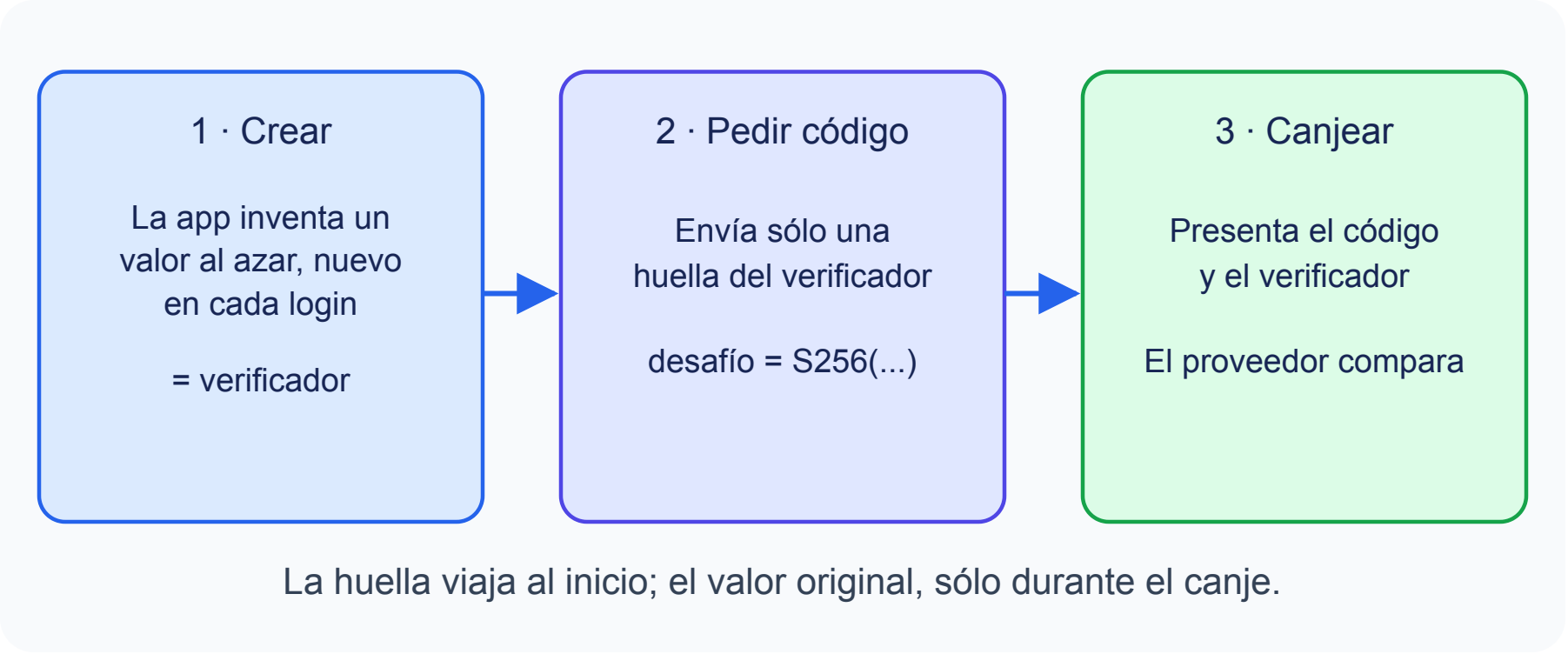
Paso 4: intercambiar el código



El canje es **entre servidores**: nuestra app y el proveedor. El navegador no lo ve.

PKCE: probar que inicié este pedido

PKCE = *Proof Key for Code Exchange*: una prueba para el canje del código.



Aunque alguien vea el **código**, no puede canjearlo sin el **verificador**.

¿Qué protege `state`?

Vincula la respuesta con **esta solicitud y este navegador**.

Si el valor recibido no coincide, la app **rechaza el callback**.

No poner datos sensibles sin protección dentro de `state`.

`state`, PKCE y `nonce` cumplen funciones relacionadas, **no idénticas**.

Paso 5: usar el access token

```
GET /api/recurso-protegido HTTP/1.1
Authorization: Bearer ACCESS_TOKEN
```

La API valida el token y el acceso solicitado.

Bearer: quien posee el token puede presentarlo. Debe viajar y almacenarse con cuidado.

Un token no es necesariamente un JWT

- **Access token:** credencial para acceder a una API; puede ser opaca o tener otro formato.
- **JWT:** formato de token con claims; no es un protocolo de autorización.
- **ID token (OIDC):** token de identidad; siempre es un JWT.

No usar un ID token como access token de una API.

OAuth no alcanza para «Iniciar sesión»

OAuth responde: **¿puede este cliente acceder a este recurso?**

No garantiza, por sí solo, una afirmación estandarizada sobre **quién se autenticó**.

Para «Ingresar con un proveedor» usamos **OpenID Connect sobre OAuth 2.0**.

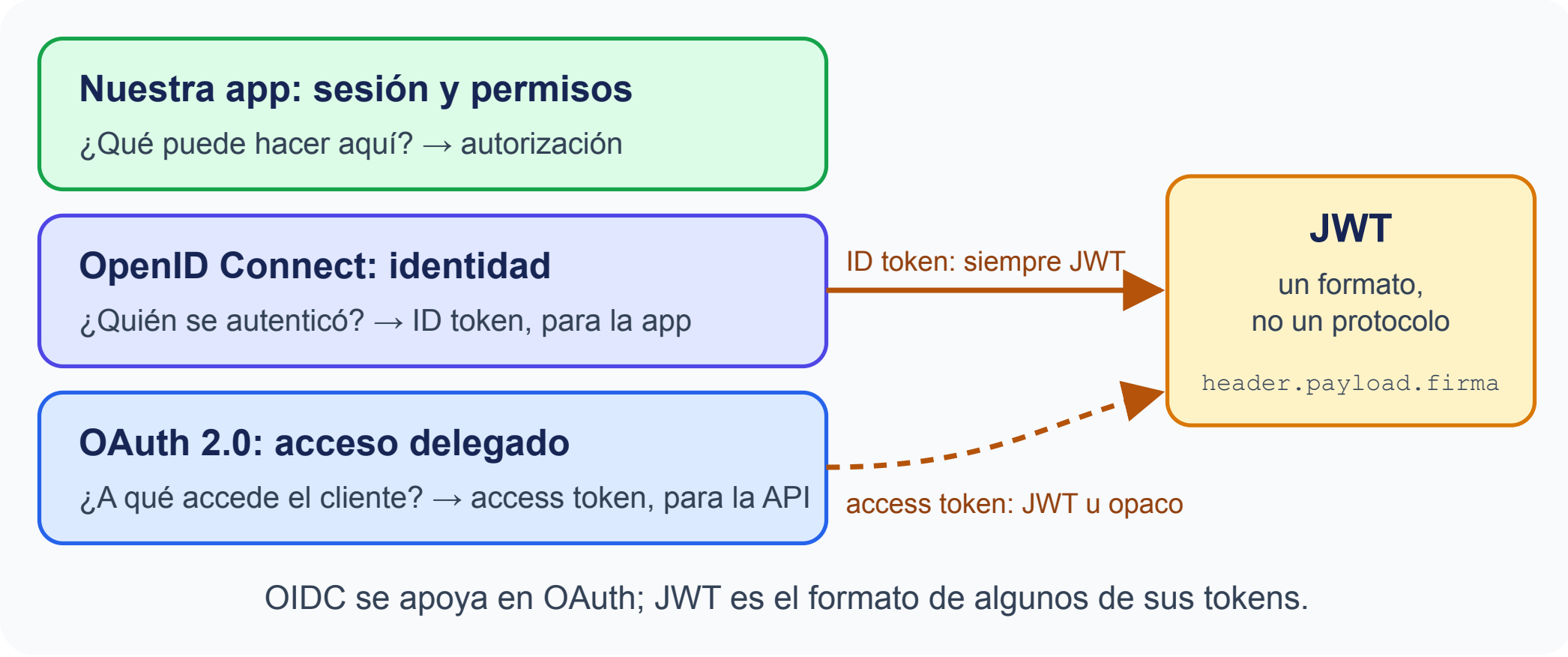
¿Qué agrega OpenID Connect?

OAuth delega acceso; **OIDC agrega una capa de identidad**.

- `scope=openid` indica una solicitud OIDC.
- El proveedor devuelve un **ID token** para la aplicación.
- Se puede solicitar información de perfil (`email`, `profile`).
- El cliente debe **validar** esa información antes de iniciar su sesión.

OIDC no decide los permisos del hospital.

OAuth, OIDC y JWT: ¿quién es quién?



Dos protocolos, un formato y nuestras reglas.

Así funciona «Ingresar con un proveedor»

```
1. La app inicia Authorization Code + PKCE con scope=openid.
2. El proveedor autentica a la persona.
3. La app canjea el código y recibe un ID token.
4. Valida el ID token y vincula (iss, sub) con un usuario local.
5. La app crea su propia sesión para continuar navegando.
```

La sesión local no es el access token del proveedor.

¿Qué afirma un ID token?

Ejemplo de **payload ilustrativo**, no un token completo:

```
{
  "iss": "https://identidad.example",
  "sub": "persona-42",
  "aud": "app-hospital",
  "exp": 1790000000,
  "nonce": "valor-del-intento"
}
```

Emisor · sujeto · destinatario · vencimiento · intento

`aud` (*audience*) = **para quién es** el token. Quien lo recibe comprueba que diga su nombre; si no, lo rechaza aunque la firma sea válida.

Antes de creerle al ID token

La aplicación valida, con una biblioteca adecuada:

1. **Firma** y algoritmo esperado.
2. `iss` : proveedor configurado.
3. `aud` : esta aplicación, no otra.
4. `exp` y demás tiempos relevantes.
5. `nonce` , si se usó en la solicitud.

Leer el JSON no comprueba su autenticidad.

Una identidad externa no es un email

Para vincular la cuenta externa con una cuenta local, usar:

```
(issuer, subject) → usuario local
```

`iss` identifica al proveedor; `sub`, a la persona **en ese proveedor**.

El email puede cambiar y no debe ser la clave única de identidad.

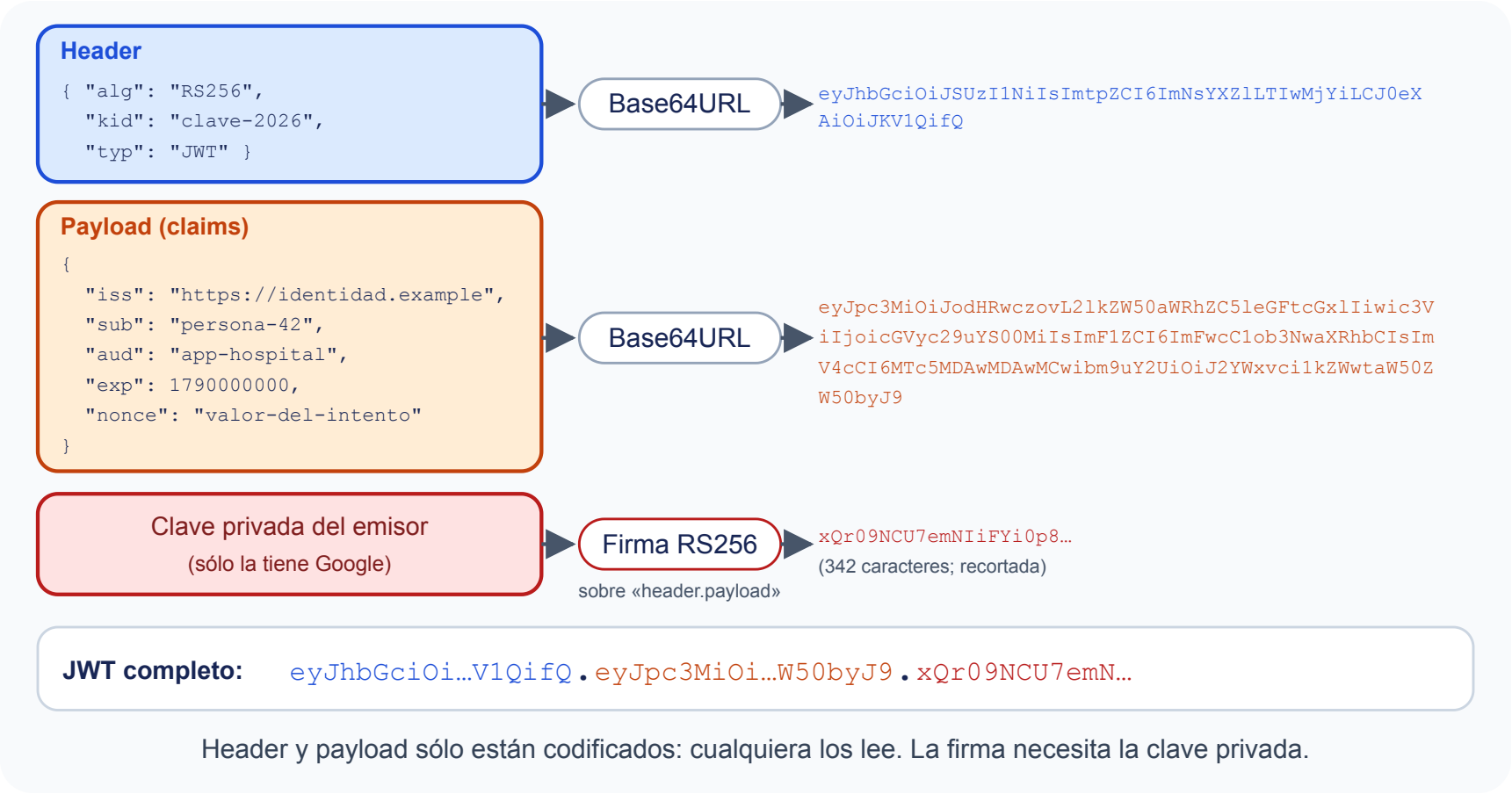
La aplicación crea su propia sesión

```
ID token validado → (iss, sub) → usuario local → sesión de la app
```

- El proveedor acredita una identidad externa.
- La aplicación decide cómo vincularla y qué datos conservar.
- En cada operación protegida, comprueba los permisos del usuario local.
- Cerrar sesión borra **la sesión de la app**, no la del proveedor.

Inicio de sesión y autorización son pasos distintos.

JWT: tres partes con funciones distintas



Decodificar las dos primeras partes no equivale a verificar.

Firmado no significa cifrado

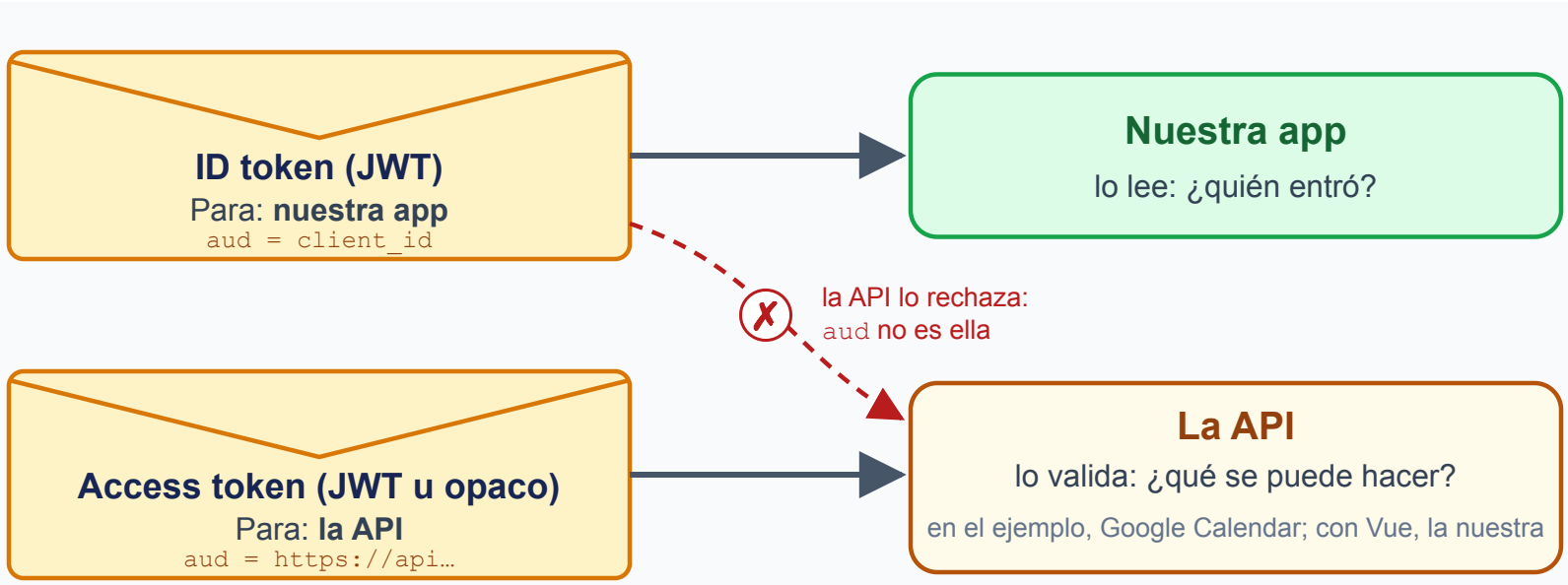
El payload de un JWT **firmado** se puede leer: no poner allí contraseñas ni datos sensibles innecesarios.

- **JWS:** protege integridad y origen; no oculta el contenido.
- **JWE:** cifra el contenido; es otro mecanismo.

La firma importa sólo si se valida con claves y reglas confiables.

¿Dónde aparece JWT en una API?

Un **access token** puede ser JWT, aunque también puede ser opaco.



Mismo formato posible; propósito y destinatario diferentes.

Nuestra API no acepta «cualquier JWT válido»

Cuando **nuestra API** recibe un access token JWT (como el que emitirá Flask en la clase de Vue), valida:

- firma y algoritmo permitidos;
- `iss`: un emisor en el que confía;
- `aud`: **nuestra API**;
- `exp` y otros límites temporales;
- alcance y permisos necesarios para **esta operación**.

Un token válido para otra API (como la de Google) **no autoriza la nuestra**.

La decisión de negocio sigue aquí

```
Proveedor: «Se autenticó la cuenta (iss, sub)»
Aplicación: «Esta cuenta corresponde al usuario 42»
Aplicación: «El usuario 42 puede / no puede autorizar esta solicitud»
```

Ni el proveedor ni el token adivinan las reglas de nuestro dominio.

401 y 403 en nuestra API

Situación	Respuesta orientativa
Falta autenticación válida	`401 Unauthorized`
Identidad válida, permiso insuficiente	`403 Forbidden`

La respuesta final debe evitar revelar información sensible sobre recursos ajenos.

Ahora, una aplicación real

Flask + Authlib + Flask-Session + Google (OIDC).

```
GET  /auth/login                  → Google            (Paso 1)
GET  /auth/callback               ← code + state      (Pasos 3 y 4)
GET  /solicitudes                 requiere sesión de la app
POST /solicitudes/<id>/aprobar    requiere permiso local
GET  /agenda                      usa el access token (Paso 5)
POST /auth/logout                 cierra la sesión de la app
```

El mismo flujo de las diapositivas, ahora con código.

Configurar el cliente OIDC

```
SCOPES = "openid email profile"
SCOPE_CALENDARIO = "https://www.googleapis.com/auth/calendar.readonly"

oauth = OAuth()
oauth.register(
    name="proveedor",
    server_metadata_url=app.config["OIDC_DISCOVERY_URL"],
    client_id=app.config["OIDC_CLIENT_ID"],
    client_secret=app.config["OIDC_CLIENT_SECRET"],  # en .env, nunca en git
    client_kwargs={"scope": SCOPES + " " + SCOPE_CALENDARIO,
                   "code_challenge_method": "S256"},
)
```

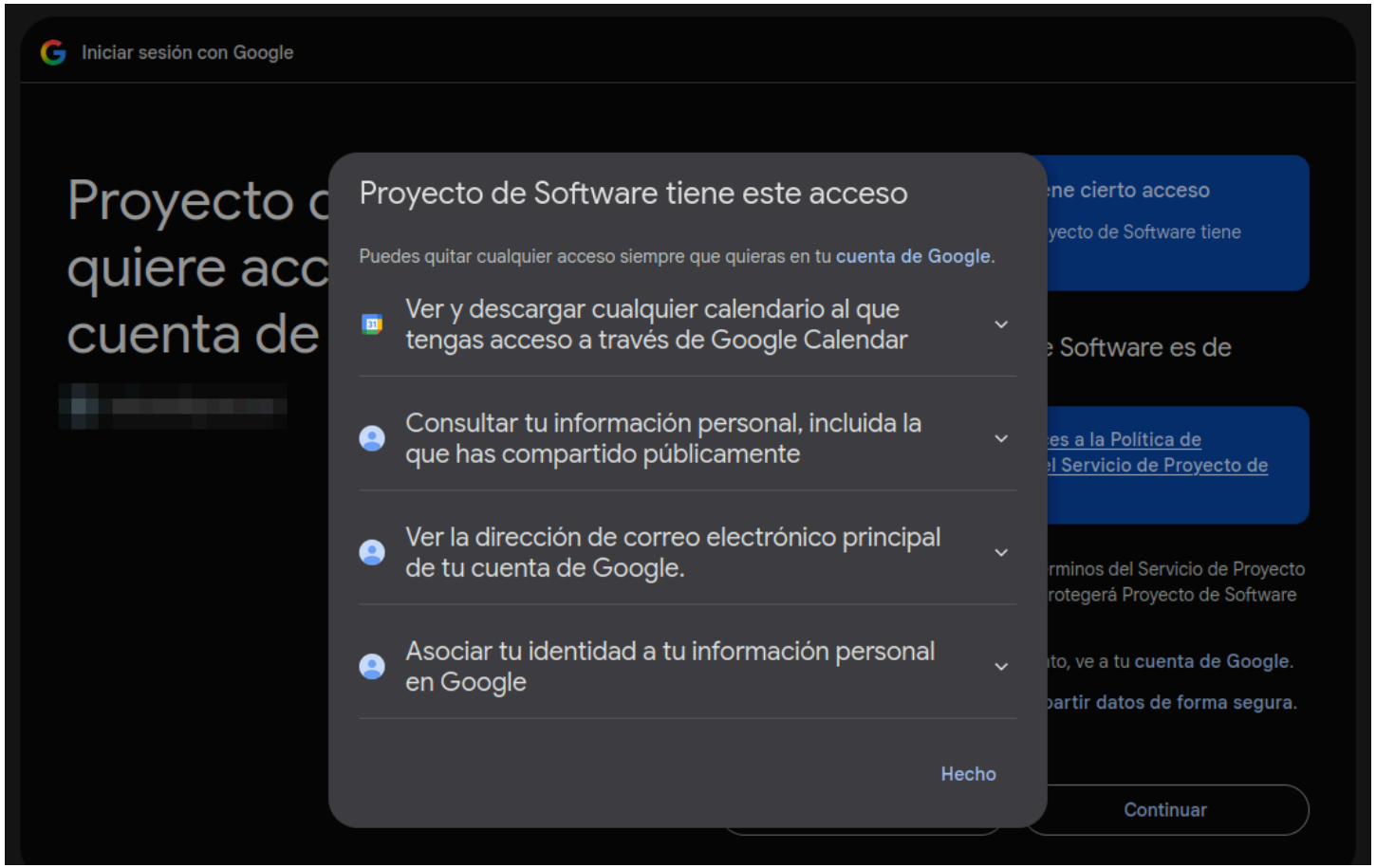
El **discovery** de Google informa endpoints, emisor y claves: no se escriben a mano.

Login: URL de callback y redirección

```python
@bp.get("/login")
def login():
    # Debe coincidir EXACTAMENTE con la URI registrada en Google:
    # http://127.0.0.1:5004/auth/callback
    redirect_uri = url_for("auth.callback", _external=True)
    return oauth.proveedor.authorize_redirect(redirect_uri, prompt="consent")
```

Authlib arma la URL con `client_id`, `redirect_uri`, `scope`, `state`, `nonce` y `code_challenge`, y guarda en la sesión lo necesario para el retorno.

Paso 2 en detalle: qué pide nuestra app



Nuestros scopes, traducidos: ¿cuál es un recurso y cuáles son identidad?

Callback: del ID token a la sesión propia

```python
@bp.get("/callback")
def callback():
    try:
        # Verifica state, canjea code + verifier y valida el ID token
        token = oauth.proveedor.authorize_access_token()
    except OAuthError as error:
        if error.error == "access_denied":  # la persona canceló:
            return redirect(url_for("inicio"))  # no es un error
        return render_template("error.html", error=error), 400  # state…
    claims = token["userinfo"]              # claims ya validados
    session.clear()
    session["usuario"] = {"iss": claims["iss"], "sub": claims["sub"],
                          "nombre": claims.get("name")}
    return redirect(url_for("solicitudes.index"))
```

Simplificado. La sesión vive **en el servidor**: la cookie sólo lleva un identificador.

Proteger la página y la operación

```
@bp.get("")
@login_requerido                          # sin sesión → vuelve al inicio
def index(): ...

@bp.post("/<int:solicitud_id>/aprobar")
@login_requerido
def aprobar(solicitud_id):
    if not permisos.puede_aprobar(session["usuario"], jefes):  # (iss, sub)
        abort(403)                        # autenticado no es autorizado
    ...

@bp.post("/logout")
def logout():
    session.clear()                       # cierra la app, no la cuenta de Google
```

Usar el access token ante otra API

```
token = access_token_actual()        # en el servidor, no en la cookie
if SCOPE_CALENDARIO not in token["scope"].split():
    return render_template("agenda.html", otorgado=False)

respuesta = oauth.proveedor.get(EVENTOS_URL, token=token)
# GET https://www.googleapis.com/calendar/v3/calendars/primary/events
# Authorization: Bearer ya29.a0A…
```

Es un access token **para la API de Google**: nuestra app no lo acepta como credencial propia.

¿Qué dato viaja en cada paso?

Momento	¿Qué vemos?	¿Quién lo recibe?
Autorización	`state`, scope y desafío PKCE	Proveedor
Retorno	código y `state`	Nuestra app
Canje	código y verificador	Proveedor
Ingreso OIDC	ID token	Nuestra app
Llamada a la API	access token	API de Google (Calendar)

El código viaja por el navegador; los tokens, no.

Un ingreso y una petición válidos

Para repasar el flujo:

1. ¿Coincide el `state` que vuelve?
2. ¿Se canjea el código con el verificador correcto?
3. ¿Para quién es el **ID token**?
4. ¿Para quién es el **access token**?
5. ¿Qué permiso local requiere la operación?

No alcanza con que «funcione el login».

¿Dónde se rechaza cada caso?

Caso	Lo rechaza	Nuestra app responde
Consentimiento cancelado	La persona, en Google	Vuelve al inicio: **no es un error**
`state` distinto	Nuestra app, en el callback	**400** `mismatching_state`
Código con otro verificador	Google, en el canje	**400** `invalid_grant`
ID token con `aud` ajena	Authlib, al validar el ID token	**400**
Cuenta válida sin permiso	Nuestra app, al aprobar	**403**

`access_denied`, `invalid_grant`… son avisos de OAuth, no códigos HTTP. **400**: el callback no es válido. **403**: identificado, pero sin permiso.

¿Aceptarías esta URL de retorno?

Una app registra `https://app.example/callback`, pero pide redirigir a otra URL.

¿Debe aceptarse? No: el proveedor debe validar la URI registrada.

Una redirección abierta puede filtrar códigos o tokens.

¿Qué pasa si roban el código?

Alguien ve el código que vuelve por el navegador.

¿Qué evita que lo intercambie? PKCE: necesita el `code_verifier` del intento original; un cliente confidencial además se autentica cuando corresponde.

HTTPS y una URI de retorno correcta siguen siendo necesarios.

¿Puede aprobar una solicitud ajena?

La cuenta externa se autenticó correctamente, pero intenta aprobar una solicitud ajena.

¿Qué debe pasar? La aplicación rechaza la operación según sus reglas; el inicio de sesión externo no concede ese permiso.

¿Verdadero o falso?

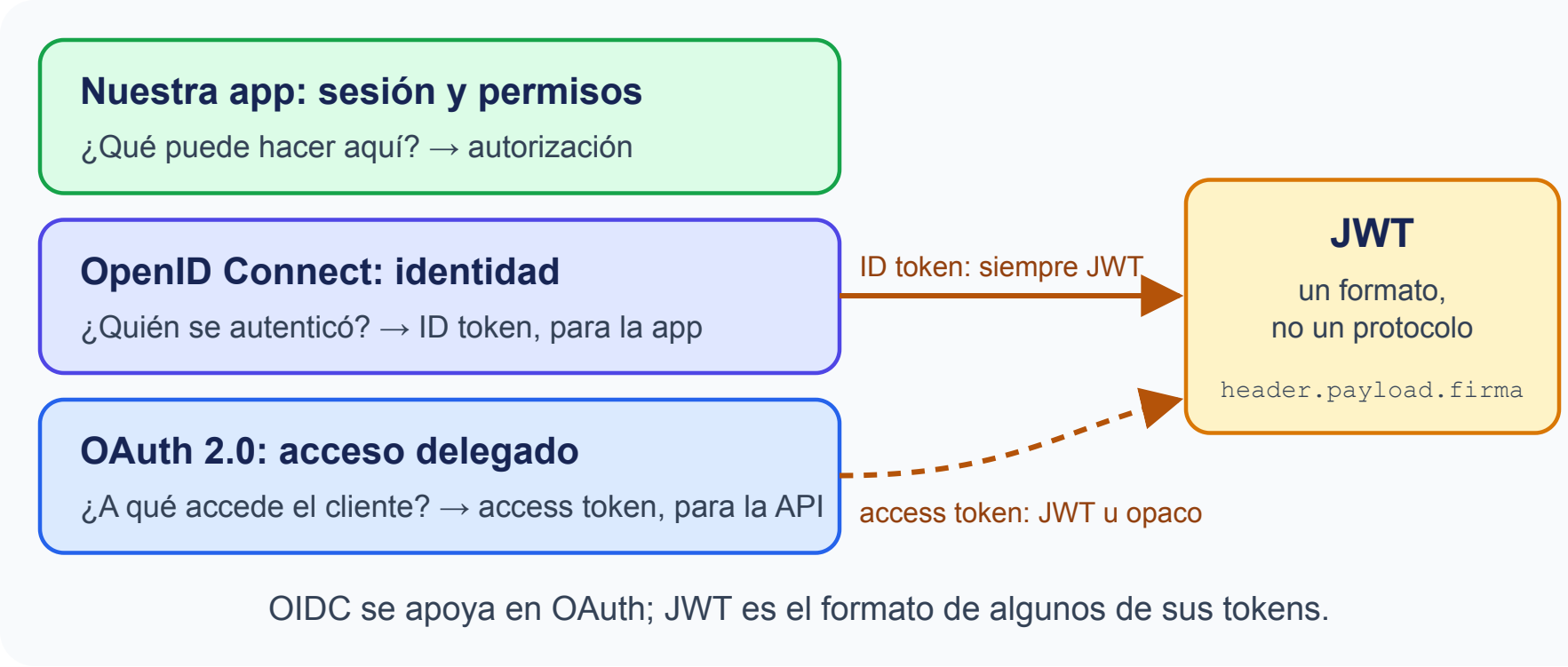
Decidamos **verdadero o falso**:

1. «OAuth 2.0 es un protocolo de inicio de sesión».
2. «Un access token siempre es un JWT».
3. «El proveedor puede autenticar; nuestra app conserva las reglas de negocio».
4. «Un `client_id` es una contraseña».

Revisemos las respuestas

1. **Falso:** para identidad estandarizada, OIDC.
2. **Falso:** el formato del access token no está fijado aquí.
3. **Verdadero:** autenticarse afuera no es autorizarse en el negocio.
4. **Falso:** identifica al cliente; no se trata como secreto.

Cómo se conectan estas piezas



Integrar identidad y acceso no reemplaza nuestras reglas.

Cinco ideas clave

- **OAuth 2.0:** delegar acceso limitado.
- **OpenID Connect:** autenticar e identificar sobre OAuth.
- **JWT:** formato de claims; puede servir a propósitos distintos.
- **ID token no es access token:** cambian destinatario y uso.
- **Nuestra aplicación:** vincula usuarios y decide permisos sobre cada recurso.

Próxima clase: ¿y si el frontend es Vue?

```
1. Vue     navega a /auth/login (no es un fetch)
2. Flask ⇄ Google: el flujo OIDC de hoy
3. Flask   valida y emite NUESTRO JWT
4. Vue     usa ese JWT con nuestra API
```

El flujo de hoy no cambia: cambia cómo recibe Vue la sesión.

Fuentes para profundizar

- OAuth 2.0 — RFC 6749
- Seguridad OAuth — RFC 9700
- PKCE — RFC 7636
- OpenID Connect Core 1.0
- JWT — RFC 7519
- Perfil JWT para access tokens — RFC 9068